42. Операторы сравнения

В языке C++ есть 6 операторов сравнения:

Оператор	Символ	Пример	Операция
Больше	`>`	x > y	true, если x больше y, в противном случае — false
Меньше	`<`	x < y	true, если x меньше y, в противном случае — false
Больше или равно	`>=`	x >= y	true, если x больше/равно y, в противном случае — false
Меньше или равно	`<=`	x <= y	true, если x меньше/равно y, в противном случае — false
Равно	`==`	x == y	true, если x равно y, в противном случае — false
Не равно	`!=`	x != y	true, если x не равно y, в противном случае — false

Вы уже могли их видеть в коде. Они довольно простые. Каждый из этих операторов вычисляется в **логическое значение** true (1) или false (0).

Вот несколько примеров использования этих операторов на практике:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y;
    std::cin >> y;

    if (x == y)
        std::cout << x << " equals " << y << "\n";
    if (x != y)
        std::cout << x << " does not equal " << y << "\n";
    if (x > y)
        std::cout << x << " is greater than " << y << "\n";
    if (x < y)
        std::cout << x << " is less than " << y << "\n";
    if (x >= y)
        std::cout << x << " is greater than or equal to " << y << "\n";
    if (x <= y)
        std::cout << x << " is less than or equal to " << y << "\n";

    return 0;
}
```

Результат выполнения программы:

```
Enter an integer: 4
Enter another integer: 5
4 does not equal 5
4 is less than 5
4 is less than or equal to 5
```

Всё просто!

Сравнение чисел типа с плавающей точкой

Сравнение значений **типа с плавающей точкой** с помощью любого из этих операторов — дело опасное. Почему? Из-за тех самых небольших **ошибок округления**, которые могут привести к неожиданным результатам. Например:

```
#include <iostream>

int main()
{
    double d1(100 - 99.99); // должно быть 0.01
    double d2(10 - 9.99); // должно быть 0.01

    if (d1 == d2)
        std::cout << "d1 == d2" << "\n";
    else if (d1 > d2)
        std::cout << "d1 > d2" << "\n";
    else if (d1 < d2)
        std::cout << "d1 < d2" << "\n";

    return 0;
}
```

Вот так раз:

```
d1 > d2
```

В вышеприведенной программе `d1 = 0.01000000000000511`, а `d2 = 0.009999999999997868`. Значения обоих этих чисел очень близки к `0.1`, но `d1` больше `d2`. Они не равны.

Иногда сравнение чисел типа с плавающей точкой бывает неизбежным. В таком случае следует использовать операторы `>`, `<`, `>=` и `<=` только если значения этих чисел сильно отличаются друг от друга. А вот если два операнда почти равны, то результат уже может быть неожиданный. В вышеприведенном примере последствия неправильного результата незначительны, а вот с оператором равенства дела обстоят хуже, так как даже при самой маленькой неточности результат сразу меняется на противоположный ожидаемому. Не рекомендуется использовать операторы `==` или `!=` с числами типа с плавающей точкой. Вместо них следует использовать функцию, которая вычисляет, насколько *эквивалентны* эти два значения. Если разницей между ними можно пренебречь, то мы считаем их равными. Значение разницы между числами, которой можно пренебречь, называется **эпсилоном**. Оно, обычно, небольшое (например, `0.0000001`).

Очень часто начинающие разработчики пытаются писать свои собственные функции определения равенства чисел:

```
#include <cmath> // для функции fabs()

bool isAlmostEqual(double a, double b, double epsilon)
{
    // если абсолютное значение a - b меньше значения эпсилона, то тогда a и b можно считать равными
    return fabs(a - b) <= epsilon;
}
```

> ***Примечание:*** Функция fabs() — это функция из **заголовочного файла** cmath, которая возвращает абсолютное значение (модуль) параметра. `fabs(a - b)` возвращает положительное число как разницу между `a` и `b`.

Функция isAlmostEqual() из примера, приведенного выше, сравнивает разницу `(a - b)` и эпсилон, вычисляя, таким образом, можно ли считать эти числа равными. Если разница между `a` и `b` очень мала, то функция возвращает true.

Хоть это и рабочий вариант, но он не идеален. Эпсилон 0.00001 подходит для чисел около 1.0, но будет слишком большим для чисел типа 0.0000001 и слишком малым для чисел типа 10000. Это означает, что каждый раз при вызове функции нам нужно будет выбирать наиболее соответствующий входным данным функции эпсилон.

Дональд Кнут, известный учёный, предложил следующий способ в своей книге «Искусство программирования, том 2: Получисленные алгоритмы» (1968):

```
#include <cmath> // для функции fabs()

// Возвращает true, если разница между a и b в пределах процента эпсилона
bool approximatelyEqual(double a, double b, double epsilon)
{
    return fabs(a - b) <= ((fabs(a) < fabs(b) ? fabs(b) : fabs(a)) * epsilon);
}
```

Здесь, вместо использования эпсилона как абсолютного числа, мы используем его как умножитель, чтобы подстроиться под входные данные.

Рассмотрим детально, как работает функция approximatelyEqual(). Слева от оператора `<=` абсолютное значение `(a - b)` сообщает нам разницу между `a` и `b` (положительное число). Справа от `<=` нам нужно вычислить эпсилон, т.е. наибольшее значение разности чисел, которое мы готовы принять. Для этого алгоритм выбирает большее из чисел `a` и `b` (как приблизительный показатель общей величины чисел), а затем умножает его на эпсилон. В этой функции эпсилон представляет собой процентное соотношение. Например, если разница между числами `a` и `b` находится в пределах 1% (больше или меньше), то мы вводим эпсилон 1% (1% = 1/100 = 0.01). Его значение можно легко регулировать, в зависимости от обстоятельств (например, 0.01% = эпсилон 0.0001). Чтобы сделать неравенство `(!=)` вместо равенства — просто вызовите эту функцию, используя **логический оператор НЕ** `(!)`, чтобы перевернуть результат:

```
if (!approximatelyEqual(a, b, 0.001))
    std::cout << a << " is not equal to " << b << "\n";
```

Но и функция approximatelyEqual() тоже не идеальна, особенно, когда дело доходит до чисел, близких к нулю:

```
#include <iostream>
#include <cmath> // для функции fabs()

// Возвращает true, если разница между a и b в пределах процента эпсилона
bool approximatelyEqual(double a, double b, double epsilon)
{
    return fabs(a - b) <= ((fabs(a) < fabs(b) ? fabs(b) : fabs(a)) * epsilon);
}

int main()
{
    // Значение d очень близко к 1.0, но, из-за ошибок округления, чуть меньше 1.0
    double d = 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1;

    // Во-первых, давайте сравним значение d (почти 1.0) с 1.0
    std::cout << approximatelyEqual(d, 1.0, 1e-8) << "\n";

    // Во-вторых, давайте сравним значение d - 1.0 (почти 0.0) с 0.0
    std::cout << approximatelyEqual(d - 1.0, 0.0, 1e-8) << "\n";
}
```

Возможно, вы удивитесь, но результат:

```
1
0
```

Второй вызов не сработал так, как ожидалось. Математика просто ломается, когда дело доходит до нулей.

Но и этого можно избежать, используя как абсолютный эпсилон (то, что мы делали в первом способе), так и относительный (способ Кнута) вместе:

```
// Возвращает true, если разница между a и b меньше absEpsilon или в пределах relEpsilon
bool approximatelyEqualAbsRel(double a, double b, double absEpsilon, double relEpsilon)
{
    // Проверяем числа на равенство их друг другу - это нужно в тех случаях, когда сравниваемые числа является нулевыми или "
    double diff = fabs(a - b);
    if (diff <= absEpsilon)
        return true;

    // В противном случае возвращаемся к алгоритму Кнута
    return diff <= ((fabs(a) < fabs(b) ? fabs(b) : fabs(a)) * relEpsilon);
}
```

Здесь мы добавили новый параметр — `absEpsilon`. Сначала мы сравниваем `a` и `b` с `absEpsilon`, который должен быть задан как очень маленькое число (например, `1e-12`). Таким образом, мы решаем случаи, когда `a` и `b` — нулевые значения или близки к нулю. Если это не так, то мы возвращаемся к алгоритму Кнута.

Протестируем:

```
#include <iostream>
#include <cmath> // для функции fabs()

// Возвращает true, если разница между a и b в пределах процента эпсилона
bool approximatelyEqual(double a, double b, double epsilon)
{
    return fabs(a - b) <= ((fabs(a) < fabs(b) ? fabs(b) : fabs(a)) * epsilon);
}

// Возвращает true, если разница между a и b меньше absEpsilon или в пределах relEpsilon
bool approximatelyEqualAbsRel(double a, double b, double absEpsilon, double relEpsilon)
{
    // Проверяем числа на равенство их друг другу - это нужно в случаях, когда сравниваемые числа является нулевыми или около
    double diff = fabs(a - b);
    if (diff <= absEpsilon)
        return true;

    // В противном случае, возвращаемся к алгоритму Кнута
    return diff <= ((fabs(a) < fabs(b) ? fabs(b) : fabs(a)) * relEpsilon);
}

int main()
{
    // Значение d очень близко к 1.0, но, из-за ошибок округления, чуть меньше 1.0
    double d = 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1;

    std::cout << approximatelyEqual(d, 1.0, 1e-8) << "\n";    // сравниваем "почти 1.0" с 1.0
    std::cout << approximatelyEqual(d - 1.0, 0.0, 1e-8) << "\n"; // сравниваем "почти 0.0" с 0.0
    std::cout << approximatelyEqualAbsRel(d - 1.0, 0.0, 1e-12, 1e-8) << "\n"; // сравниваем "почти 0.0" с 0.0
}
```

Результат:

```
1
0
1
```

С удачно подобранным `absEpsilon`, функция approximatelyEqualAbsRel() обрабатывает близкие к нулю и нулевые значения корректно.

Сравнение чисел типа с плавающей точкой — сложная тема, и нет одного идеального алгоритма, который подойдет в любой ситуации. Однако для большинства случаев, с которыми вы будете сталкиваться, функции approximatelyEqualAbsRel() должно быть достаточно.